

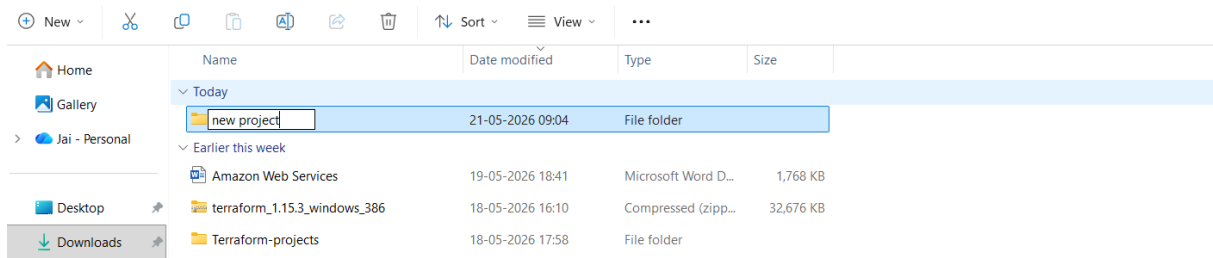
TERRAFORM CONFIGURATION FILES

Terraform uses configuration files with .tf extension to define and manage infrastructure resources using HCL (HashiCorp Configuration Language).

HANDS-ON FOR TERRAFORM CONFIGURATION FILES

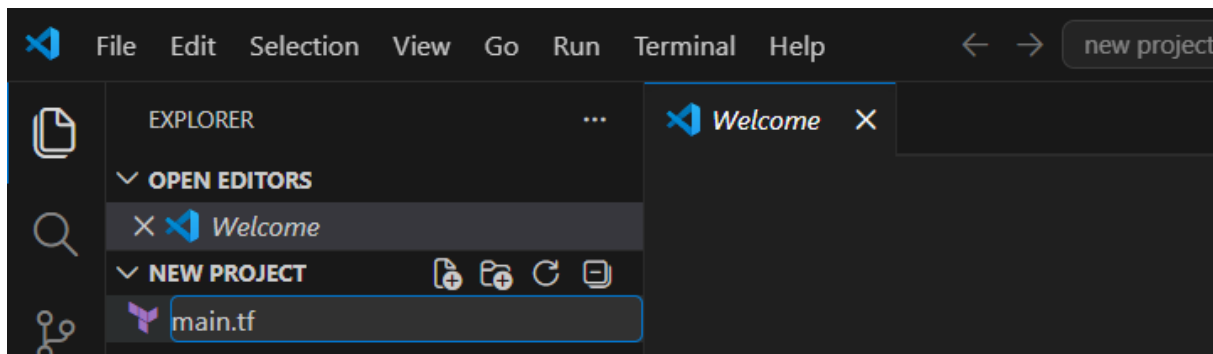
Screenshot: Create Terraform Project Folder

- Open Downloads folder in local system
- Create a new folder for Terraform project



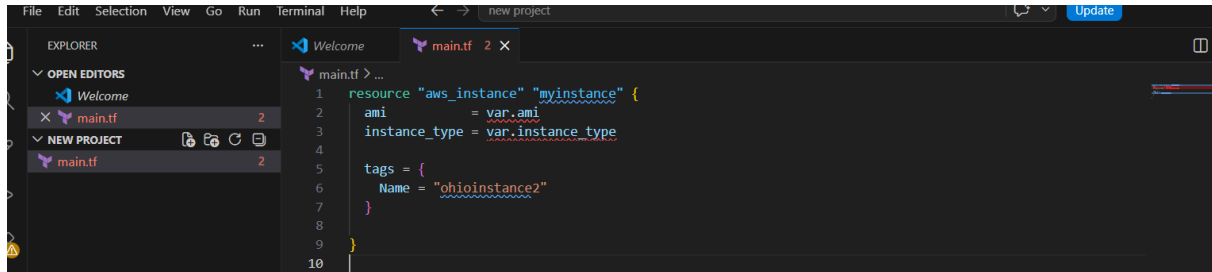
Screenshot: Create Main Terraform File and Create Main Terraform File

- Open Visual Studio Code
- Click File → Open Folder
- Select the new Terraform project folder from Downloads
- Create a new file in VS Code
- Enter file name as main.tf
- Main Terraform configuration file created successfully



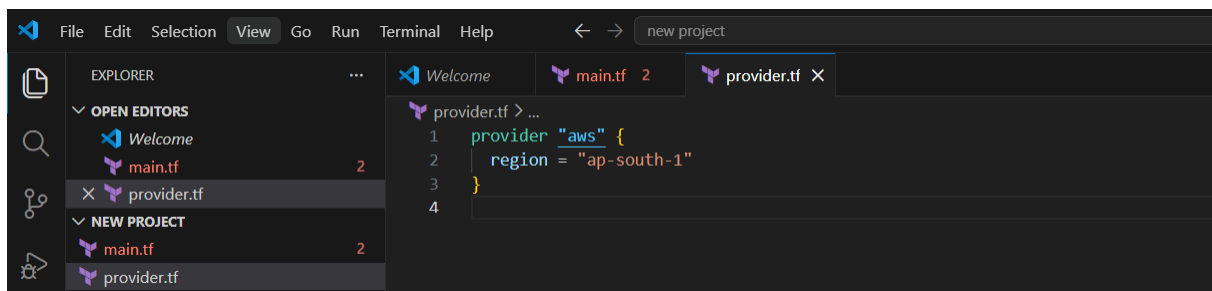
Screenshot: Add EC2 Resource Configuration in Main File

- Paste the Terraform EC2 instance configuration code into main.tf
- Save the main.tf file successfully



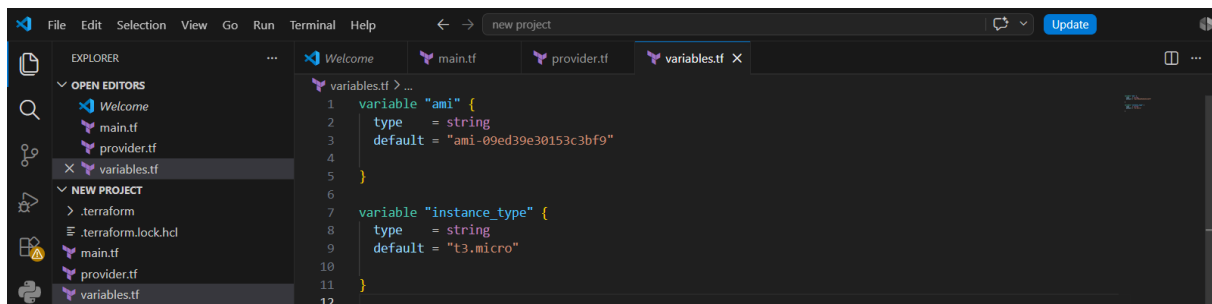
Screenshot: Create Provider Configuration File

- Create a new file named provider.tf
- Paste the Terraform provider configuration code into the file
- Save the provider.tf file successfully



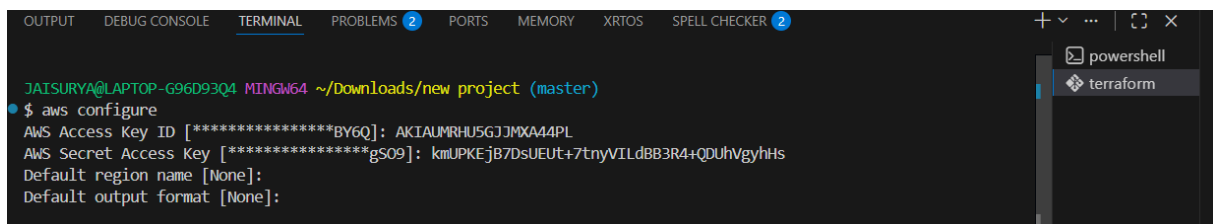
Screenshot: Create Variables File

- Create a new file named variables.tf
- Paste the variable declaration code into the file
- Save the variables.tf file successfully



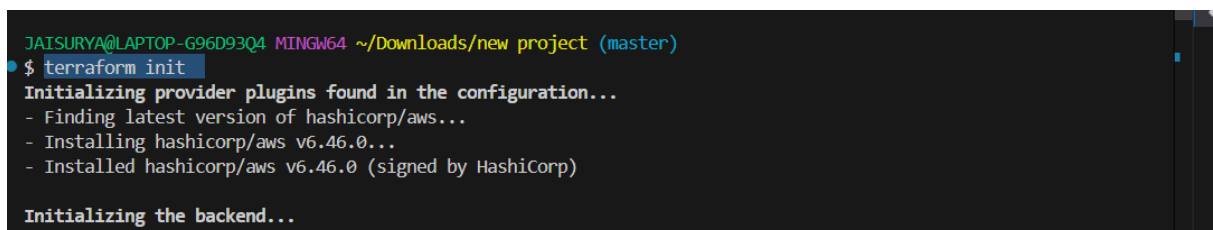
Screenshot: Configure AWS Credentials

- Open terminal in VS Code
- Execute aws configure
- Enter AWS Access Key ID
- Enter AWS Secret Access Key
- Enter default AWS region name
- AWS CLI configuration completed successfully

A screenshot of a Visual Studio Code terminal window. The terminal shows the command prompt for a user named JAISURYA on a laptop with ID G96D93Q4, using a MINGW64 shell. The user has navigated to the directory ~/Downloads/new project (master). They have executed the command 'aws configure'. The terminal shows the prompts and inputs for 'AWS Access Key ID' (masked with asterisks), 'AWS Secret Access Key' (masked with asterisks), 'Default region name' (set to None), and 'Default output format' (set to None). The configuration is completed successfully. On the right side of the terminal window, there is a file explorer showing a folder named 'terraform'.

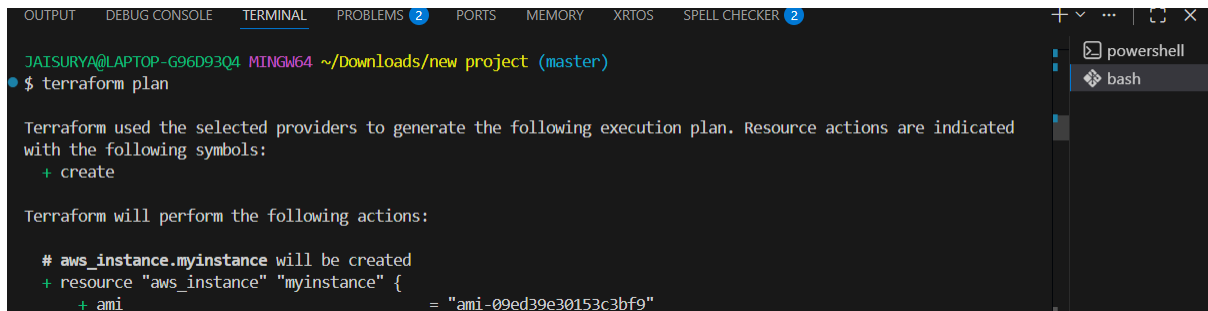
Screenshot: Initialize Terraform Project

- Execute terraform init
- Terraform initializes the working directory
- Required provider plugins downloaded successfully
- Terraform initialization completed successfully

A screenshot of a Visual Studio Code terminal window. The terminal shows the command prompt for a user named JAISURYA on a laptop with ID G96D93Q4, using a MINGW64 shell. The user has navigated to the directory ~/Downloads/new project (master). They have executed the command 'terraform init'. The terminal shows the output of the command, which includes 'Initializing provider plugins found in the configuration...', 'Finding latest version of hashicorp/aws...', 'Installing hashicorp/aws v6.46.0...', and 'Installed hashicorp/aws v6.46.0 (signed by HashiCorp)'. The terminal also shows 'Initializing the backend...'. The command is completed successfully.

Screenshot: Execute Terraform Plan

- Execute terraform plan
- Terraform displays execution plan
- Shows resources that will be created using variable values



```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/new project (master)
$ terraform plan

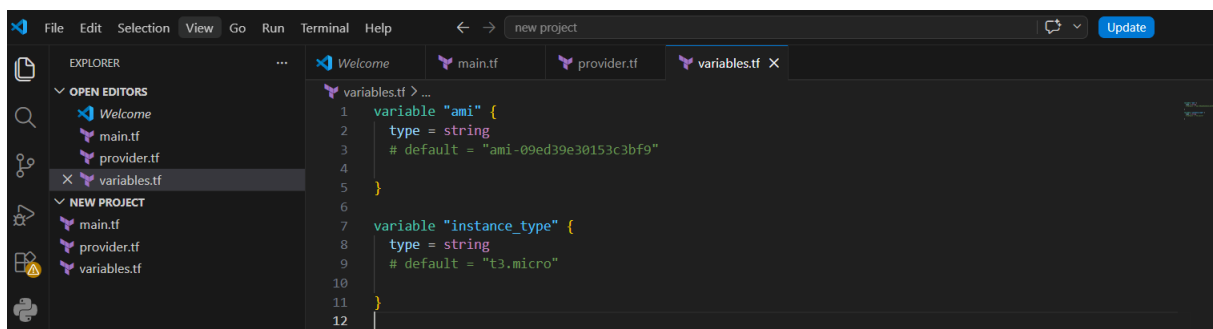
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.myinstance will be created
+ resource "aws_instance" "myinstance" {
+   ami           = "ami-09ed39e30153c3bf9"
```

Screenshot: Comment Default Variable Values

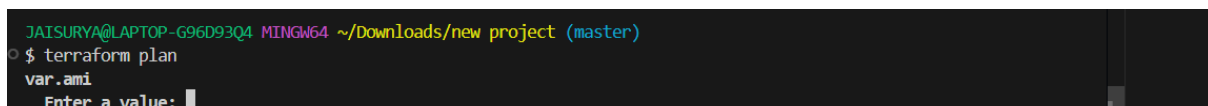
- Use # symbol before AMI ID and instance type default values in variables.tf
- Default values commented successfully in the variables file



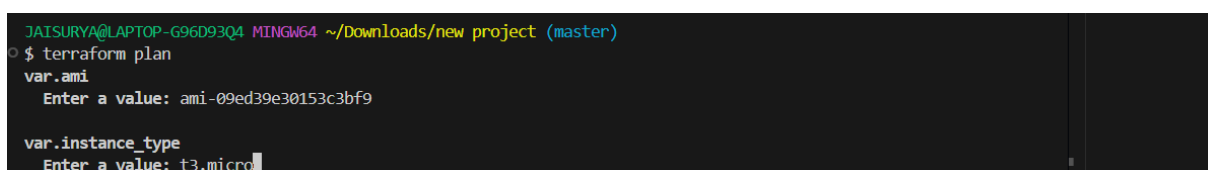
```
variables.tf > ...
1  variable "ami" {
2    type = string
3    # default = "ami-09ed39e30153c3bf9"
4  }
5
6
7  variable "instance_type" {
8    type = string
9    # default = "t3.micro"
10 }
11
12
```

Screenshot: Manual Variable Input During Terraform Plan

- Execute terraform plan
- Terraform prompts for manual input of AMI ID
- Enter the AMI ID value
- Terraform prompts for instance type
- Enter the instance type value
- Execution plan generated successfully for EC2 instance creation



```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/new project (master)
$ terraform plan
var.ami
Enter a value: 
```



```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/new project (master)
$ terraform plan
var.ami
Enter a value: ami-09ed39e30153c3bf9

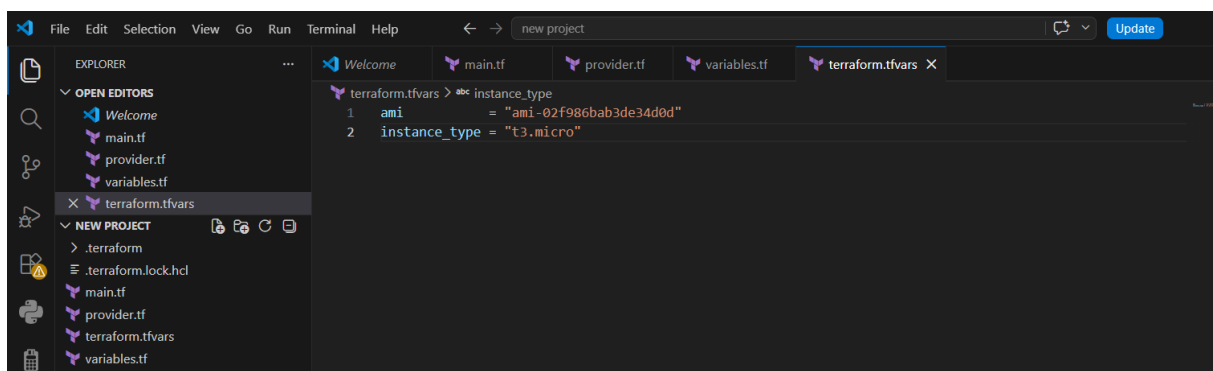
var.instance_type
Enter a value: t3.micro
```

```
Plan: 1 to add, 0 to change, 0 to destroy.
```

```
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
```

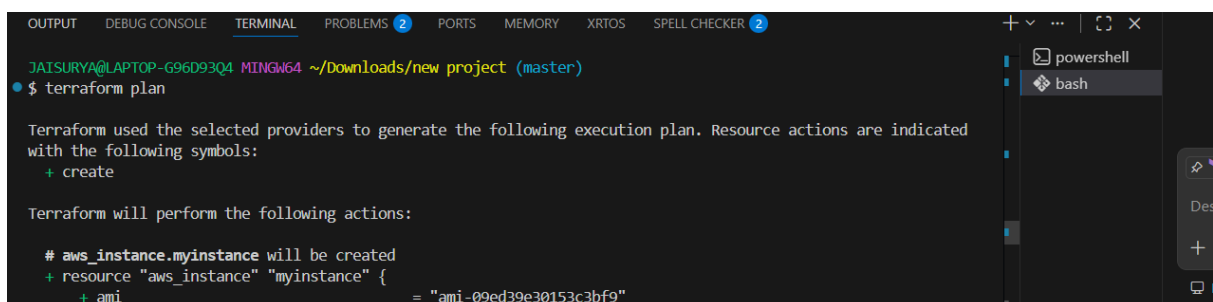
Screenshot: Create Terraform Variable Values File

- Create a new file named terraform.tfvars
- Paste the AMI ID and instance type variable values into the file
- Save the terraform.tfvars file successfully



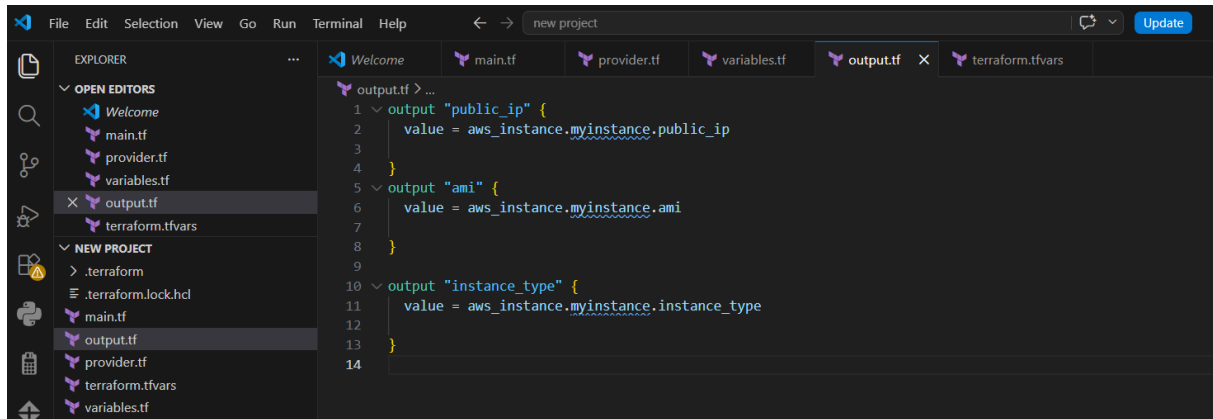
Screenshot: Automatic Variable Loading Using terraform.tfvars

- Execute terraform plan
- Terraform automatically reads variable values from terraform.tfvars file
- No manual AMI ID or instance type entry required
- Execution plan generated successfully even after commenting default values in variables.tf



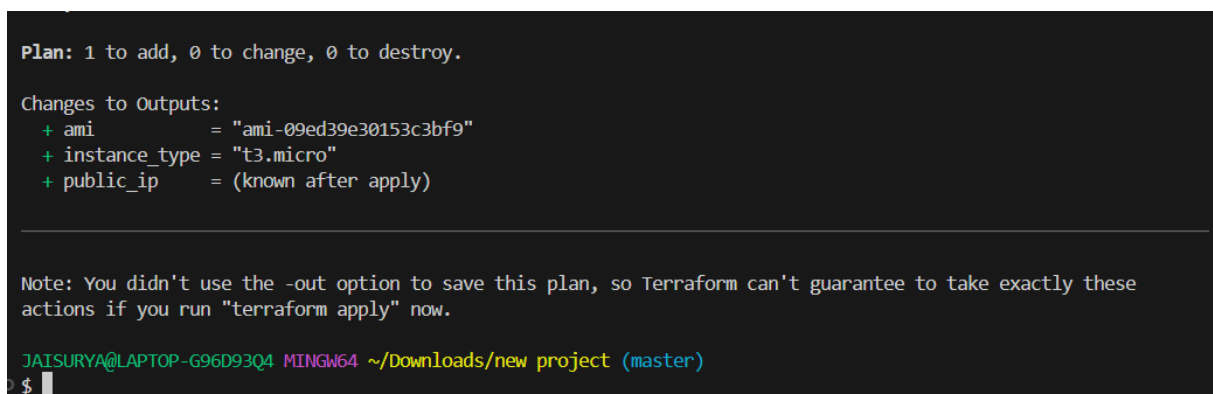
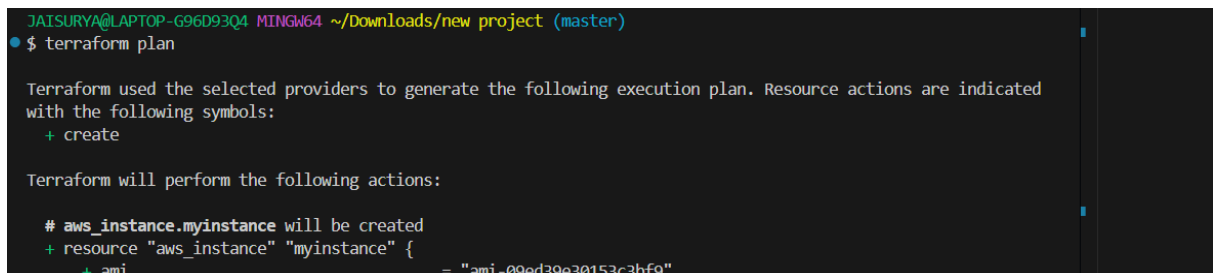
Screenshot: Create Terraform Output File

- Create a new file named outputs.tf
- Paste the Terraform output configuration code into the file
- Save the outputs.tf file successfully



Screenshot: Terraform Plan with Output Values

- Execute terraform plan
- Terraform displays AMI ID and instance type values
- Public IP output shown as (known after apply) because the instance is not created yet



Screenshot: Terraform Apply with Output Values

- Execute terraform apply
- Type yes to approve infrastructure creation
- Terraform creates the EC2 instance successfully
- Output values displayed after apply execution
- AMI ID, instance type, and public IP address shown successfully

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/new project (master)
$ terraform apply
```

```
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
```

```
+ create
```

```
Terraform will perform the following actions:
```

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

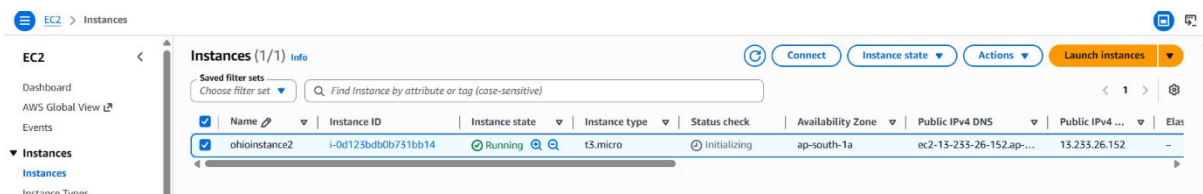
Outputs:

```
ami = "ami-09ed39e30153c3bf9"
instance_type = "t3.micro"
public_ip = "13.233.26.152"
```

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/new project (master)
$
```

Screenshot: Verify EC2 Instance in AWS Console

- Open AWS Management Console
- Go to EC2 Dashboard
- Open Instances section
- EC2 instance created successfully using Terraform configuration files



Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elas
ohioinstance2	i-0d123bdc0b751bb14	Running	t3.micro	Initializing	ap-south-1a	ec2-13-233-26-152.ap...	13.233.26.152	--

Screenshot: Verify Public IP Address

- Go to EC2 Dashboard
- Check the Public IP address of the created instance
- Public IP matches the Terraform output value displayed after apply execution

Public IPv4 address

 13.233.26.152 | [open address](#) 

Screenshot: Destroy Terraform Infrastructure

- Execute terraform destroy
- Terraform displays resources that will be deleted

```
JAISURYA@LAPTOP-G96D93Q4 MINGW64 ~/Downloads/new project (master)
$ terraform destroy --auto-approve
aws_instance.myinstance: Refreshing state... [id=i-0d123bdb0b731bb14]

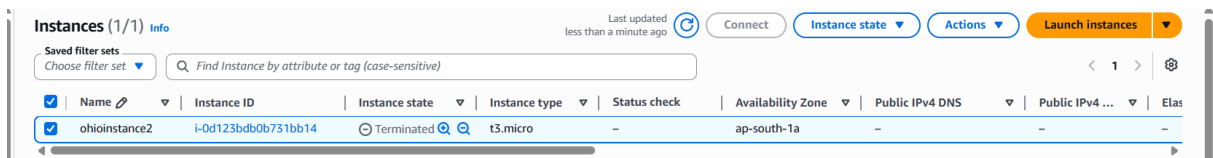
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
  - destroy

Terraform will perform the following actions:

# aws_instance.myinstance will be destroyed
- resource "aws_instance" "myinstance" {
  - ami              = "ami-09ed39e30153c3bf9" -> null
  - arn              = "arn:aws:ec2:ap-south-1:301804611986:instance/i-0d123bdb0b731bb14"
```

Screenshot: Verify Resource Deletion in AWS Console

- Go to EC2 Dashboard
- Open Instances section
- EC2 instance terminated successfully after Terraform destroy execution



The screenshot shows the AWS Management Console 'Instances' page. The instance 'ohioinstance2' with ID 'i-0d123bdb0b731bb14' is shown in a 'Terminated' state. The table below summarizes the instance details.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elas
ohioinstance2	i-0d123bdb0b731bb14	Terminated	t3.micro	-	ap-south-1a	-	-	-